

Software Design Patterns

Assignment 2

PC Factory

Abstract Factory and Factory Method in Java

Student: Temujin Gonchar

Group: SE 2518

University: AITU

Repository: github.com/AltezA-jet/SDP-Assignment-2

1. Introduction and domain

The project models **PC assembly**. A user chooses a PC family (Budget, Gaming, Professional or Extreme) and the program builds a configuration from a CPU, a GPU and a cooling system. For the configuration it calculates the total power consumption, an overall performance score and whether the cooling is balanced with the CPU.

The goal of the assignment is to apply two creational patterns: **Abstract Factory**, which creates whole families of compatible products, and **Factory Method**, which lets subclasses decide which object to create. The project is written in Java and built with Maven; tests use JUnit 5.

2. Products and families

There are three abstract products, each described by an interface:

- **CPU** - `getName()`, `getPowerConsumption()`, `getPerformance()`
- **GPU** - `getName()`, `getPowerConsumption()`, `getPerformance()`
- **Cooling** - `getName()`, `getPowerConsumption()`, `getCoolingPerformance()`

There are four families. Each family has its own concrete CPU, GPU and Cooling class (12 product classes in total). The table shows power consumption in watts and the performance value of each component.

Family	CPU (W / perf)	GPU (W / perf)	Cooling (W / perf)
Budget	65 / 50	100 / 50	30 / 50
Gaming	125 / 90	250 / 90	60 / 90
Professional	180 / 100	350 / 100	80 / 100
Extreme	250 / 100	450 / 100	100 / 100

3. Abstract Factory

The pattern is implemented in the packages `factories` and `products`.

Role	Classes
Abstract factory	PCFactory with <code>createCPU()</code> , <code>createGPU()</code> , <code>createCooling()</code>
Concrete factories	BudgetPCFactory, GamingPCFactory, ProfessionalPCFactory, ExtremePCFactory
Abstract products	CPU, GPU, Cooling
Concrete products	BudgetCPU, GamingCPU, ProfessionalCPU, ExtremeCPU; the same for GPU and Cooling
Client	PCConfiguration (business logic) and PCClient

The client depends only on PCFactory and the product interfaces. PCConfiguration receives a factory in its constructor, asks it for a CPU, a GPU and a cooling system, and uses only interface methods to compute the results. It never uses the `new` keyword for a concrete component.

PCConfiguration business methods

- `calculateTotalPower()` - sum of the power consumption of the three components.

- `calculatePerformanceScore()` - average of CPU performance, GPU performance and cooling performance.
- `isBalanced()` - true when the cooling performance is at least the CPU performance.

Example for the Gaming family: total power = 125 + 250 + 60 = 435 W; performance score = (90 + 90 + 90) / 3 = 90; the cooling is balanced.

4. Factory Method

The pattern is implemented in the package `factorymethod`.

Role	Classes
Creator	<code>ComputerCreator</code> (abstract) declares the factory method <code>createComputer()</code> and uses it in <code>prepareComputer()</code>
Concrete creators	<code>BudgetComputerCreator</code> , <code>GamingComputerCreator</code> , <code>ProfessionalComputerCreator</code>
Product	<code>Computer</code> interface with <code>build()</code>
Concrete products	<code>BudgetComputer</code> , <code>GamingComputer</code> , <code>ProfessionalComputer</code>

`prepareComputer()` prints a message, calls the abstract `createComputer()`, calls `build()` on the result and prints a final message. This code is the same for every creator. Only the subclass decides which `Computer` is instantiated. This package is a separate demonstration of Factory Method; the main application flow in `Main` uses Abstract Factory.

Difference between the two patterns in this project

Factory Method creates *one* product and the choice is made by subclassing a creator. Abstract Factory creates a *family* of related products (CPU, GPU, Cooling) through one object that is passed to the client.

5. Runtime selection

The family is chosen while the program runs. `Main` reads the first command-line argument (the default is `gaming`) and passes it to `PCFactorySelector.selectFactory(String)`, which returns the matching `PCFactory`. The name is case-insensitive.

```
"budget"      -> new BudgetPCFactory()
"gaming"      -> new GamingPCFactory()
"professional" -> new ProfessionalPCFactory()
"extreme"     -> new ExtremePCFactory()
anything else -> IllegalArgumentException("Unknown PC family: ...")
```

The selected factory is passed to `PCClient.buildPC(factory)`. The client code is the same for every family; only the factory object is different.

6. Compatibility of products

Products of one configuration are compatible for the following reasons:

- A concrete factory creates the CPU, GPU and cooling of **one family only**, so components from different families cannot be mixed by the client.

- PCConfiguration gets only a PCFactory and asks it for all three components. It has no way to choose components one by one.
- Inside every family the performance levels match (Budget 50, Gaming 90, Professional and Extreme 100), so the cooling always keeps up with the CPU. isBalanced() checks this and the tests confirm it for the Budget, Gaming and Professional families.

7. Adding the Extreme family

The Extreme family was added in a separate commit (*Add Extreme PC family with all product types*) to check how easy it is to extend the design.

Change	File
Added	factories/ExtremePCFactory.java
Added	products/cpu/ExtremeCPU.java
Added	products/gpu/ExtremeGPU.java
Added	products/cooling/ExtremeCooling.java
Modified	selection/PCFactorySelector.java (one new case "extreme" branch)
Modified	selection/PCFactorySelectorTest.java (one new test)

The interfaces PCFactory, CPU, GPU, Cooling and the classes PCConfiguration, PCClient, Main and the other families did not change. A new family needs new classes and one line in the selector, and existing client code stays untouched. This follows the Open/Closed Principle.

8. Testing

Unit tests are written with JUnit 5 and are located in src/test/java/com/example/pcfactory/.

Test class	What it checks
PCFactoryProductTest	Factories create products of the correct family (Budget CPU, Gaming GPU, Professional Cooling, all Gaming products).
PCFactorySelectorTest	Runtime selection returns the right factory for budget, gaming, professional and extreme; an unknown name throws IllegalArgumentException.
PCConfigurationBusinessTest	Gaming power 435 W, Gaming performance 90, balanced cooling, Budget power 195 W.
PCConfigurationCompatibilityTest	Budget, Gaming and Professional configurations are balanced.

9. How to run

Requirements: JDK 25 (configured in `pom.xml`) and Maven.

Run the program

```
mvn compile
java -cp target/classes com.example.pcfactory.Main gaming
```

Replace `gaming` with `budget`, `professional` or `extreme`. Without an argument the `Gaming` family is used. Output for `Gaming`:

```
Building PC
CPU: Gaming CPU
GPU: Gaming GPU
Cooling: Gaming Cooling
Total power: 435 W
Performance score: 90
Balanced cooling: true
```

Run the tests

```
mvn test
```

10. Conclusion

The project applies Abstract Factory to build compatible families of PC components chosen at runtime, and Factory Method to let creator subclasses decide which computer is built. Adding the Extreme family showed that the design is open for extension: only new classes and one selector branch were needed. Business logic and client code depend on abstractions, and unit tests cover factory output, runtime selection and the calculations.

Appendix A. UML class diagram - Abstract Factory

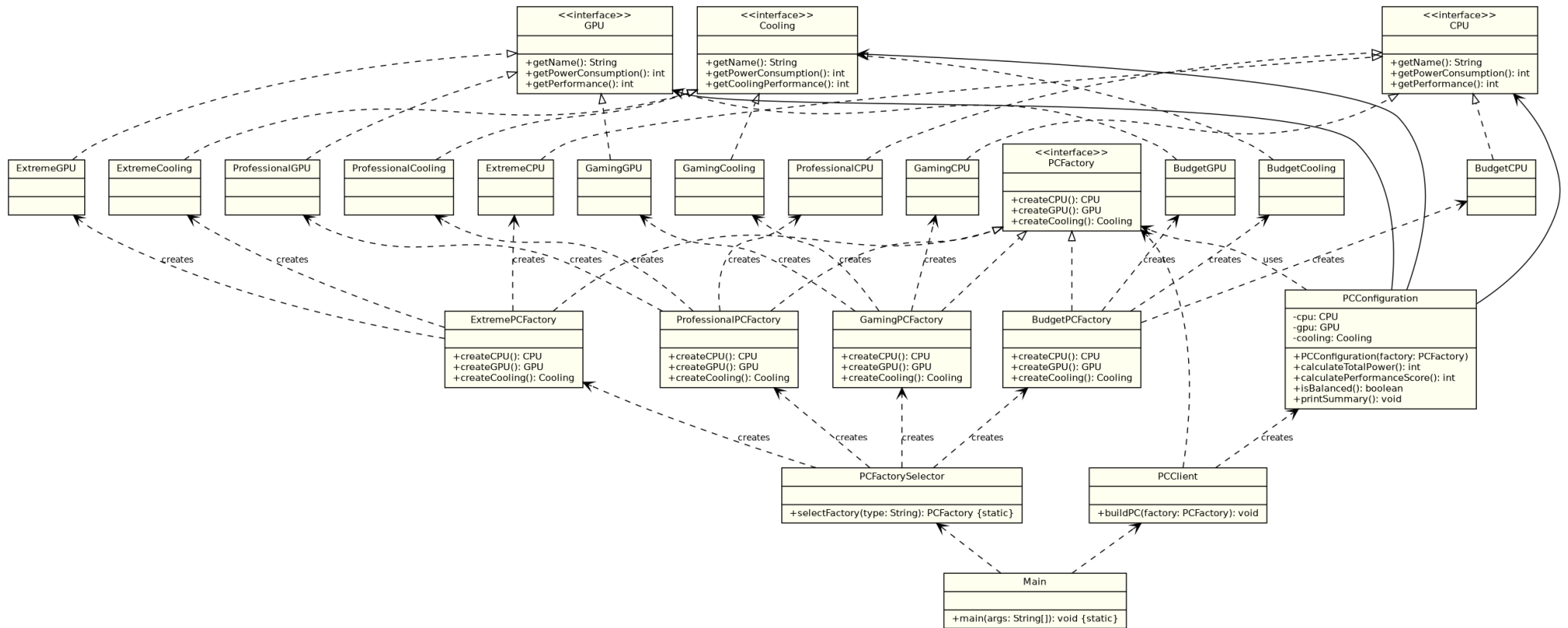


Figure 1. Abstract Factory: factories, product interfaces, product families and client classes.

Appendix B. UML class diagram - Factory Method

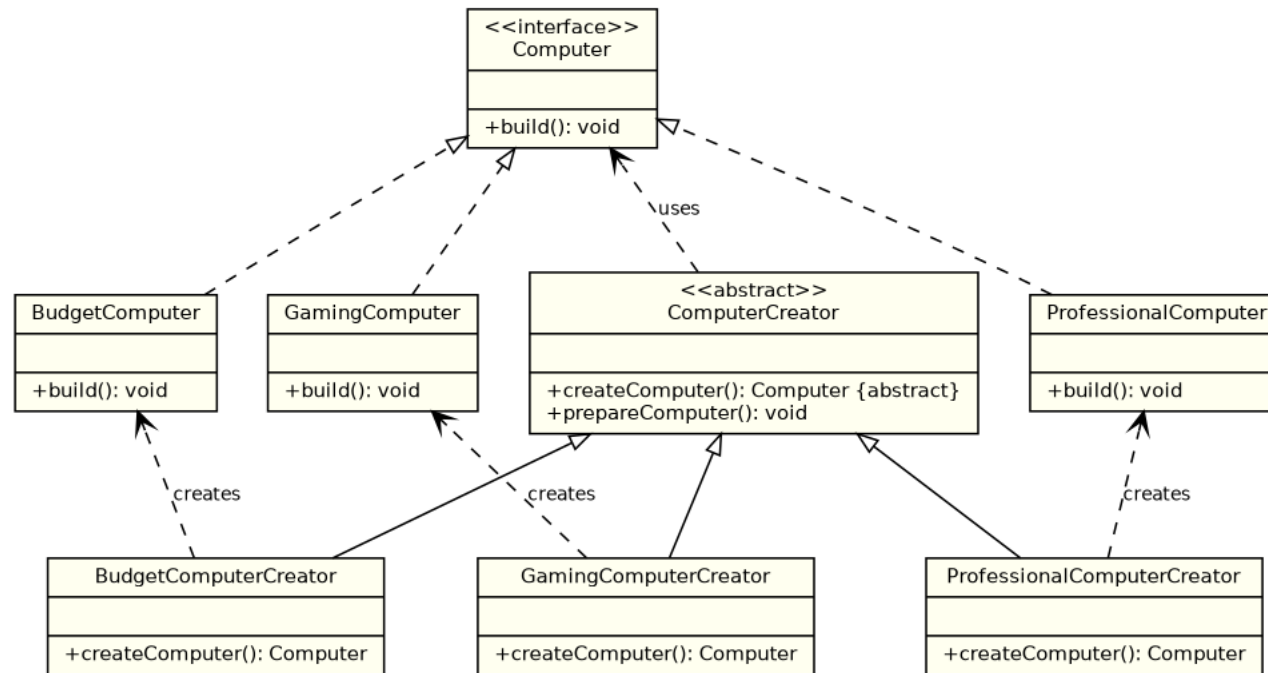


Figure 2. Factory Method: creator hierarchy and computer products.